

# MNIST Number Generation with VAE vs Langevin Sampling

## Expectation-Maximization

Mike Zeng <mzeng5@uiowa.edu>

### Introduction

This report compares variational autoencoders to an EM-like approach using Langevin sampling. We compare the latent space and the final generated images using the MNIST dataset, which contains 70,000 28x28 grayscale images of digits from 0-9.



### Design and Methods

#### Variational Autoencoder

We tested with a CVAE, a class-conditioned variational autoencoder. The encoding model takes in a linear vector of 784 floats scaled between 0 and 1 alongside a one-hot class tensor representing digits 0-9.

The variational autoencoder uses 3 layers for the encoding model of dimensions 794x256, 256x64, and 64x8. Notice the latent dimension is 8. The decoder uses 3 linear layers of 18x64, 64x256 and 256x784, with ReLU after the first two layers and a sigmoid mean output. The reported experiments use a fixed decoder log variance of 0, i.e. variance  $I$ . Initially, the model was tested with learned variance, but training became unstable and the KL term produced NaNs. However, all loss calculations still use the decoder log variance term.

Let the ELBO be defined as  $E_{q_\varphi(z|x)} [\log p_\theta(x|z)] - D_{\text{KL}} [q_\varphi(z|x) \parallel p(z)]$ . We use  $q_\varphi(z|x) = N(\mu, \text{diag}(\sigma^2))$  and  $p(z) = N(0, I)$ , so the KL term has the closed form:

$$D_{\text{KL}} [q_\varphi(z|x) \parallel p(z)] = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

The training objective is the negative ELBO, i.e. the reconstruction negative log-likelihood plus this KL term.

For the reconstruction term, we use a Gaussian observation model with decoder mean  $\mu$  and fixed variance  $I$ :

$$p_\theta(x | \mu, I) = \frac{1}{\sqrt{(2\pi)^d \det(I)}} \exp\left(-\frac{1}{2}(x - \mu)^T (I)^{-1} (x - \mu)\right)$$

$$p_\theta(x | \mu, I) = \frac{1}{\sqrt{(2\pi)^d}} \exp\left(-\frac{1}{2}(x - \mu)^2\right)$$

$$\log p_\theta(x | \mu, I) = -\frac{d}{2} \log(2\pi) - \frac{1}{2}(x - \mu)^2$$

This gives us our reconstruction term, where  $x$  is our data and  $\mu$  is taken from the decoder head. Our loss is the negative reconstruction log-likelihood plus the KL term.

## EM Algorithm

Here, we remove the encoder from the EM training loop. Rather than training a model for  $q(z|x)$ , we use an E-step-like approach to find an approximate representation of  $p(z|x)$ . The decoder is trained with the latents from the E-step, and its parameters are updated in the M-step.

### Langevin Sampling for E-step

Formally, we replace the encoder  $q_\lambda(z|x)$  with Langevin sampling to approximate the posterior  $p(z|x)$ . Given a latent representation, we can find a better representation as follows:

$$z_{t+1} = z_t + \eta \nabla \log p(z|x) + \sqrt{2\eta} \varepsilon$$

Thus, our sampling iteration becomes:

$$z_{t+1} = z_t + \eta (\nabla \log p(z) + \nabla \log p(x|z)) + \sqrt{2\eta} \varepsilon$$

If we let  $p(z)$  be a Gaussian of  $z \in N(0, I)$ , we have that

$$\log p(z) = -\frac{n}{2} \log(2\pi) - \frac{1}{2} \|z\|^2$$

$$\nabla \log p(z) = -\frac{1}{2} \nabla \|z\|^2$$

$$\nabla \log p(z) = -\frac{1}{2} (2z)$$

$$\nabla \log p(z) = -z$$

hence the first term becomes  $-z$ . The second term relies on our decoder model  $p(x|z)$ , so we obtain this through the gradient function of PyTorch. This can be done by feeding the latent through the decoder model. After our decoder model generates the mean and variance, we compute  $\log p_\theta(x|z)$  and use `autograd.grad` to obtain the gradient. Hence, our update step becomes:

$$z_{t+1} = z_t + \eta (-z + \nabla \log p(x|z)) + \sqrt{2\eta} \varepsilon$$

Note that the  $-z$  pulls the update closer to 0 while the  $\nabla \log p(x|z)$  pulls the log-likelihood of generating the data  $x$  higher.

We found that using randomly initialized vectors for the starting  $z$  resulted in the model having poor performance, so we trained the EM-approximation setup with  $z$  initialized from the encoding model with its weights frozen. We did 150 sampling steps.

### M-step Update

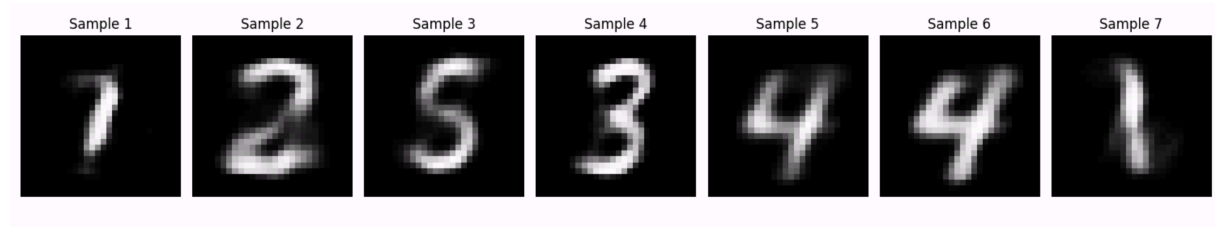
Using the finalized latent  $z$ , we update our decoder. Using the batch group  $b$ , we take the negative Gaussian likelihood of our batch as our loss. The decoder is updated using the Adam optimizer on the loss. Our M-step count was 3, so we did 3 gradient descent steps per E-step.

## Results

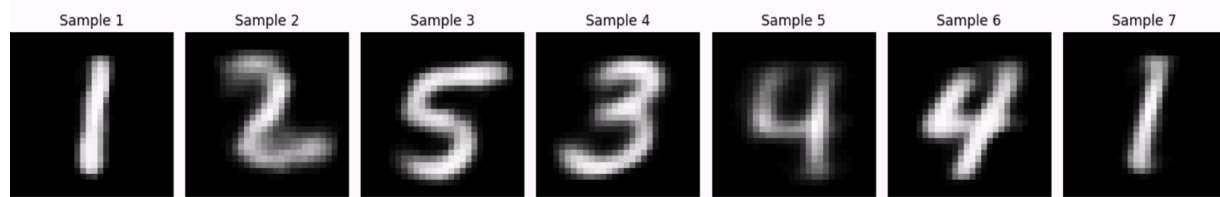
### Generated Images

We list the batch generations for each configuration. More generations can be found in <https://github.com/mzen17/CS5430-Project/examples>.

## EM-approximation with Langevin, 10000 minibatch updates with 64 image batch, 150 iterations

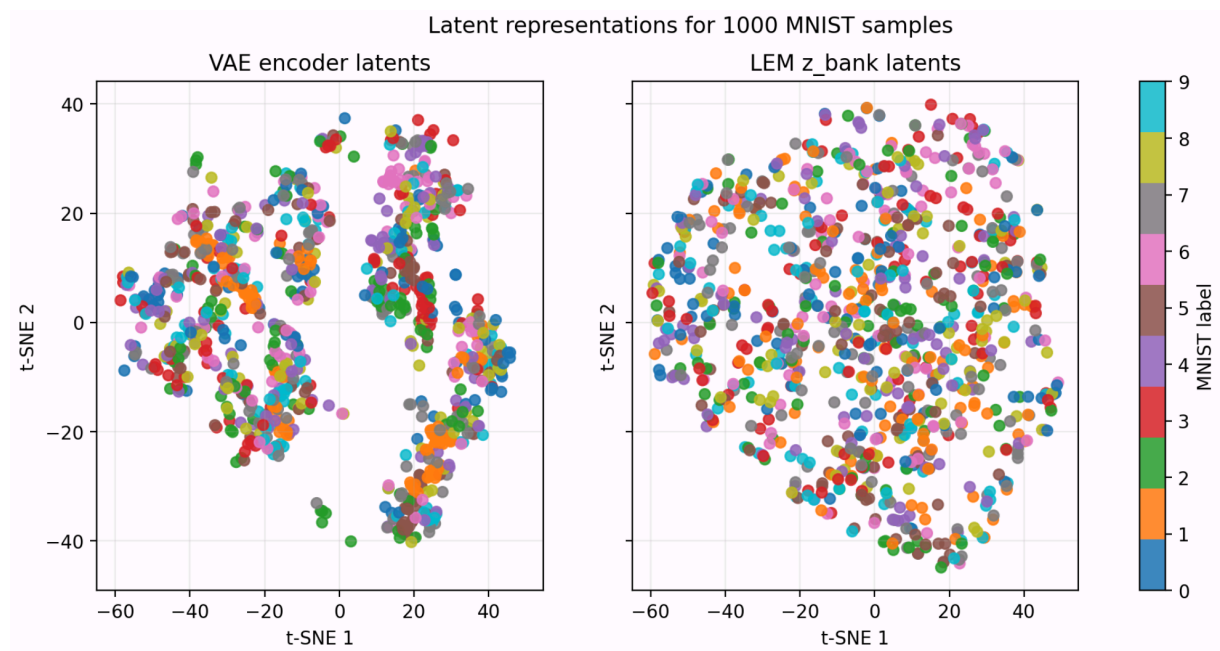


## VAE with 10000 minibatch updates, 64 image batch



## Internal Representations

We visualize 1000 latents from the finalized z-bank for the Langevin sampling approach. VAE encoder latents are generated by passing MNIST images into the encoder.



## Log likelihood Approximations

$\log p(x)$  was approximated on 50 test images using 10000 prior samples of  $z$ . The table shows the first 8 examples and the mean over all 50:

| image | label | VAE log p(x) | LEM log p(x) | LEM - VAE |
|-------|-------|--------------|--------------|-----------|
| 0     | 7     | -731.0604    | -730.8820    | 0.1784    |
| 1     | 2     | -743.6703    | -743.3235    | 0.3468    |

| image | label | VAE log p(x) | LEM log p(x) | LEM - VAE |
|-------|-------|--------------|--------------|-----------|
| 2     | 1     | -725.8842    | -725.9359    | -0.0517   |
| 3     | 0     | -735.3418    | -735.9449    | -0.6031   |
| 4     | 4     | -734.7455    | -735.2046    | -0.4590   |
| 5     | 1     | -725.4033    | -725.5439    | -0.1406   |
| 6     | 4     | -740.4456    | -740.6231    | -0.1776   |
| 7     | 9     | -740.7876    | -740.9551    | -0.1675   |
| mean  | -     | -735.8259    | -735.8308    | -0.0049   |

### Training Graphs and Time

The reported models were trained on an NVIDIA RTX 5070 Ti. The LEM model took around 3 minutes to train for 10000 steps while the VAE took about 20 seconds to train.

### Analysis

Overall, the likelihood of the data changed little between Langevin sampling and VAE encoding. The mean test log-likelihoods are nearly identical in the table, suggesting that the EM-trained decoder did not find a substantially better marginal model than the VAE decoder under this architecture and fixed-variance Gaussian observation model.

The more interesting difference is in the latent representations. The VAE latents appear more clustered by digit class, which is expected because the encoder directly learns a smooth amortized map from images to latent variables. The Langevin latents from the z-bank are less cleanly clustered. This makes sense because they are not produced by an encoder network; they are persistent latent variables refined by posterior sampling.

The picture quality of the numbers was relatively similar. Langevin sampling did produce sharper pictures in some runs, although this is not consistently visible here because generation depends on random samples. At lower numbers of training minibatches, Langevin sampling had better and sharper images. This suggests that initializing from the VAE and then refining the decoder with posterior samples can be useful early in training, but the advantage becomes smaller after enough VAE training.

The most noticeable difference was training time. Langevin sampling was around a 10x slowdown compared to training the VAE. This is expected because each minibatch requires many inner-loop gradient computations with respect to  $z$ , while the VAE obtains latents with a single encoder forward pass. Thus, the EM approximation trades amortized inference speed for a more direct but much more expensive posterior sampling procedure.

### Conclusion

The experiment shows that approximate EM with Langevin sampling is a workable alternative to standard VAE training on MNIST, but it is not clearly better in this setting. The EM-trained model produced comparable samples and nearly identical estimated test likelihoods, while requiring substantially more computation. Overall, the VAE remains the more practical method for this small image generation task.

Code: <https://github.com/mzen17/CS5430-Project>